Отчёт по лабораторной работе № 14

Расширенное программирование в shell

Савкин Дмитрий Вадимович

2026-10-12

Содержание

1 Введение **5**
1.1 Актуальность 5
1.2 Цель работы 5
1.3 Задачи 5
1.4 Объект и предмет исследования 5
1.5 Методы исследования 6
1.6 Схема выполнения работы 6

2 Теоретическая часть **7**

3 Материалы и методы **8**

4 Выполнение лабораторной работы **9**

5 Результаты **13**

6 Обсуждение результатов **14**

7 Заключение **15**

Список литературы **16**

Ответы на контрольные вопросы **17**

Список иллюстраций

4.1 Создаём рабочий каталог и пишем скрипт семафора через heredoc: mkdir -p ~/work/os/lab14, cat > task1_sem.sh « 'EOF'. Скрипт принимает время ожидания t1 и время занятия ресурса t2, использует файл-блокировку /tmp/resource.lock 9
4.2 Полный текст task1_sem.sh: цикл ожидания освобождения ресурса (с сообщениями и счётчиком w), затем touch "$LOCK" и занятие ресурса на t2 секунд, в конце rm -f "$LOCK" 10
4.3 Даём права на выполнение и тестируем: chmod +x task1_sem.sh, ./task1_sem.sh 2 4 — скрипт занимает ресурс и удерживает его 4 секунды с выводом PID ($$) 10
4.4 Пишем аналог man: cat > task2_man.sh « 'EOF'. Скрипт ищет файл справки по имени команды в /usr/share/man/man1 через ls | grep, открывает через less, при отсутствии — выводит сообщение 11
4.5 Тестируем: chmod +x task2_man.sh, ./task2_man.sh ls открывает страницу справки, ./task2_man.sh без аргумента выводит Справка для '' не найдена 11
4.6 Пишем генератор случайных букв: cat > task3_random.sh « 'EOF'. Число букв берётся первым аргументом (по умолчанию 10), каждая буква — случайный символ из алфавита через ${chars:$((RANDOM%26)):1} 12
4.7 Запускаем: ./task3_random.sh 15 и ./task3_random.sh. Замечаем опечатку в алфавите (пропущена буква j) и исправляем на месте через sed -i 's/.../.../ ' task3_random.sh, проверяем результат через grep chars 12

Список таблиц

1 Введение

1.1 Актуальность

Синхронизация процессов и работа со случайными данными — элементы, встречающиеся в более сложных системных сценариях автоматизации.

1.2 Цель работы

Написание сложных командных файлов с логическими конструкциями и циклами.

1.3 Задачи

- Написать упрощённый семафор для синхронизации процессов.
- Реализовать аналог `man` командным файлом.
- Сгенерировать случайную последовательность латинских букв через `$RANDOM`.

1.4 Объект и предмет исследования

Объект — взаимодействующие процессы; предмет — синхронизация через файл-блокировку и генерация случайных данных.

1.5 Методы исследования

Эмпирический метод: практическое выполнение заданий в виртуальной машине Fedora Linux с фиксацией результатов скриншотами и видеозаписью.

1.6 Схема выполнения работы

Работа выполнялась по стандартной схеме курса: выполнение действия в виртуальной машине, фиксация скриншотом, занесение в отчёт, публикация в git-репозиторий отдельным релизом.

2 Теоретическая часть

Расширенное программирование в shell включает работу с конкурентным доступом к общему ресурсу (простейшая синхронизация через файл-блокировку, аналог мьютекса), обёртки над системными утилитами и генерацию псевдослучайных данных через встроенную переменную `$RANDOM`.

Более подробно см. в [1].

3 Материалы и методы

Используемое ПО и оборудование: хост-машина под управлением Windows, гипервизор VirtualBox, виртуальная машина с Fedora Linux (Sway), стандартные утилиты дистрибутива.

4 Выполнение лабораторной работы

Создаём рабочий каталог и пишем скрипт семафора через heredoc: `mkdir -p ~/work/os/lab14`, `cat > task1_sem.sh << 'EOF'`. Скрипт принимает время ожидания `t1` и время занятия ресурса `t2`, использует файл-блокировку `/tmp/resource.lock`.

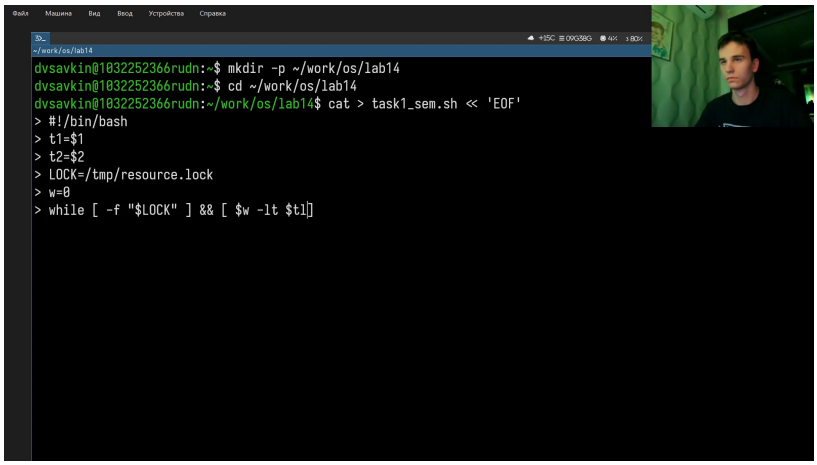


Рисунок 4.1: Создаём рабочий каталог и пишем скрипт семафора через heredoc: `mkdir -p ~/work/os/lab14`, `cat > task1_sem.sh << 'EOF'`. Скрипт принимает время ожидания `t1` и время занятия ресурса `t2`, использует файл-блокировку `/tmp/resource.lock`

Полный текст `task1_sem.sh`: цикл ожидания освобождения ресурса (с сообщениями и счётчиком `w`), затем `touch "$LOCK"` и занятие ресурса на `t2` секунд, в конце `rm -f "$LOCK"`.

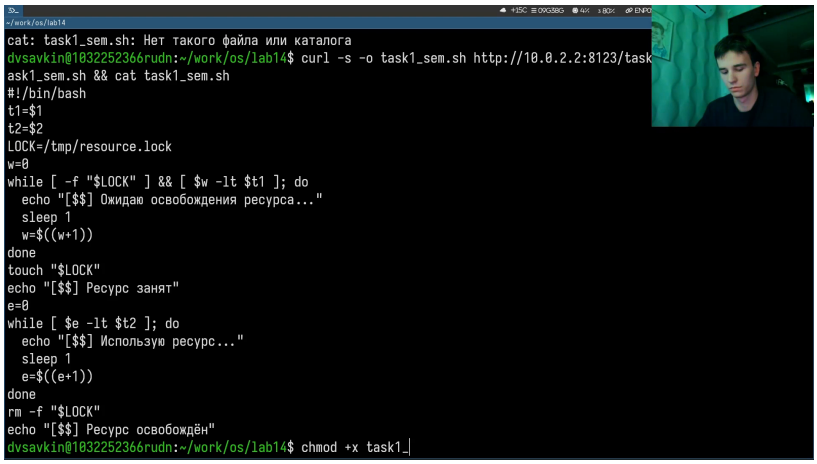


Рисунок 4.2: Полный текст `task1_sem.sh`: цикл ожидания освобождения ресурса (с сообщениями и счётчиком `w`), затем `touch "$LOCK"` и занятие ресурса на `t2` секунд, в конце `rm -f "$LOCK"`

Даём права на выполнение и тестируем: `chmod +x task1_sem.sh`, `./task1_sem.sh 2 4` — скрипт занимает ресурс и удерживает его 4 секунды с выводом PID ($$).

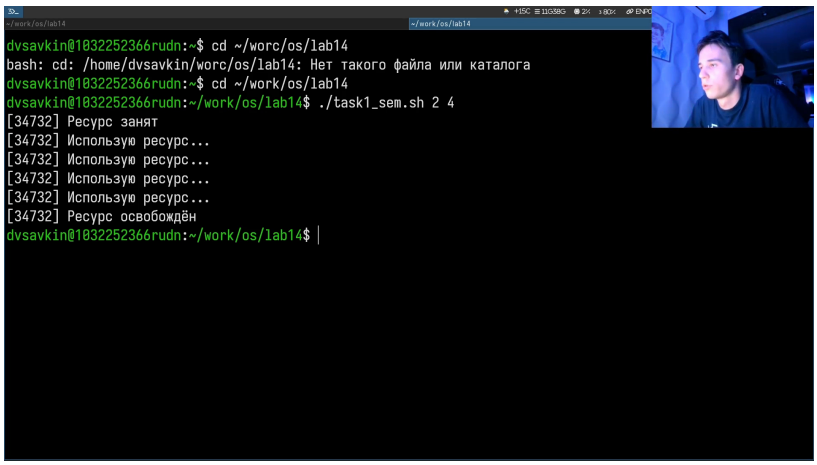


Рисунок 4.3: Даём права на выполнение и тестируем: `chmod +x task1_sem.sh`, `./task1_sem.sh 2 4` — скрипт занимает ресурс и удерживает его 4 секунды с выводом PID ($$)

Пишем аналог `man`: `cat > task2_man.sh « 'EOF'`. Скрипт ищет файл справки по имени команды в `/usr/share/man/man1` через `ls | grep`, открывает через `less`, при отсутствии — выводит сообщение.

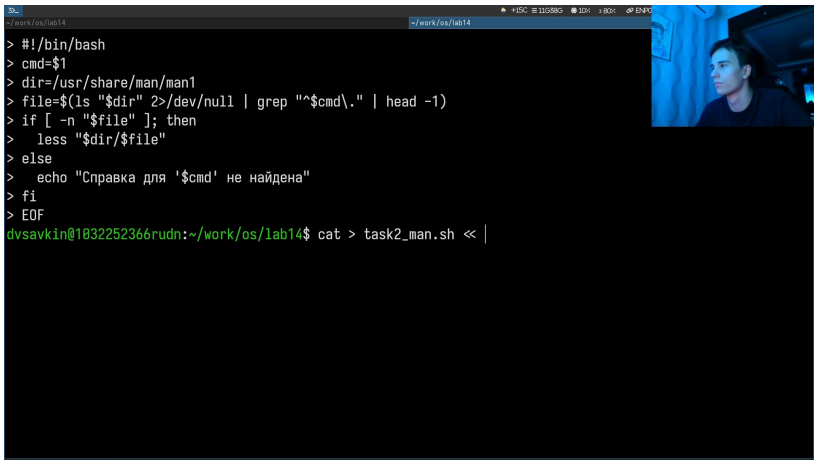


Рисунок 4.4: Пишем аналог man: `cat > task2_man.sh << 'EOF'`. Скрипт ищет файл справки по имени команды в `/usr/share/man/man1` через `ls | grep`, открывает через `less`, при отсутствии — выводит сообщение

Тестируем: `chmod +x task2_man.sh`, `./task2_man.sh ls` открывает страницу справки, `./task2_man.sh` без аргумента выводит `Справка для '' не найдена`.

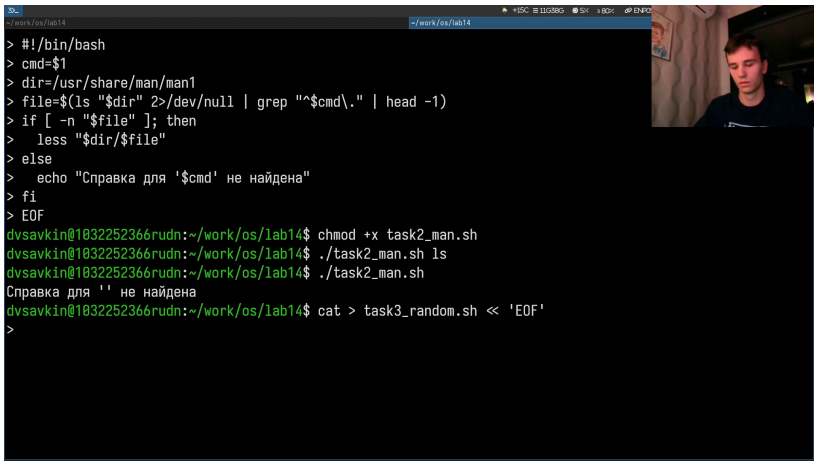


Рисунок 4.5: Тестируем: `chmod +x task2_man.sh`, `./task2_man.sh ls` открывает страницу справки, `./task2_man.sh` без аргумента выводит `Справка для '' не найдена`

Пишем генератор случайных букв: `cat > task3_random.sh << 'EOF'`. Число букв берётся первым аргументом (по умолчанию 10), каждая буква — случайный символ из алфавита через `${chars:$((RANDOM%26)):1}`.

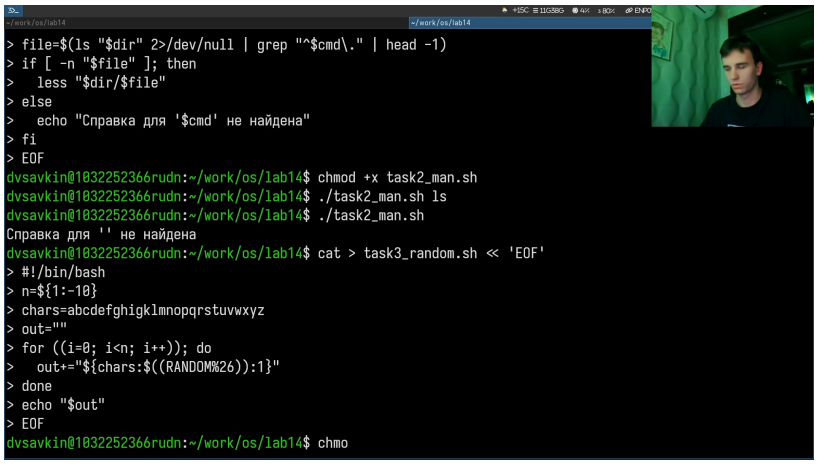


Рисунок 4.6: Пишем генератор случайных букв: `cat > task3_random.sh << 'EOF'`. Число букв берётся первым аргументом (по умолчанию 10), каждая буква — случайный символ из алфавита через `${chars:$((RANDOM%26)):1}`

Запускаем: `./task3_random.sh 15` и `./task3_random.sh`. Замечаем опечатку в алфавите (пропущена буква `j`) и исправляем на месте через `sed -i 's/.../.../ ' task3_random.sh`, проверяем результат через `grep chars`.

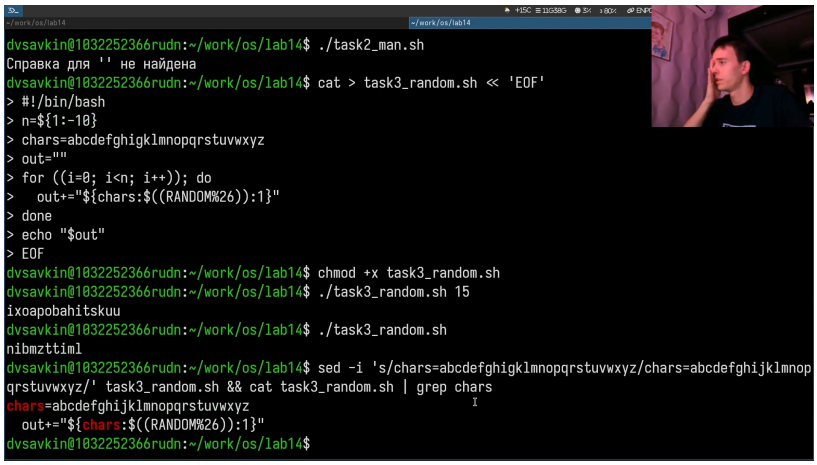


Рисунок 4.7: Запускаем: `./task3_random.sh 15` и `./task3_random.sh`. Замечаем опечатку в алфавите (пропущена буква `j`) и исправляем на месте через `sed -i 's/.../.../ ' task3_random.sh`, проверяем результат через `grep chars`

5 Результаты

В ходе выполнения работы получены следующие объективные результаты: Освоены командные файлы с циклами и условиями: упрощённый семафор на основе файла-блокировки для синхронизации процессов, обёртка над `man`, генератор случайных латинских букв через `$RANDOM`.

6 Обсуждение результатов

Семафор на основе файла-блокировки корректно предотвратил одновременный доступ к ресурсу; обнаруженная опечатка в алфавите генератора была исправлена и проверена повторным запуском.

7 Заключение

Освоены командные файлы с циклами и условиями: упрощённый семафор на основе файла-блокировки для синхронизации процессов, обёртка над `man`, генератор случайных латинских букв через `$RANDOM`.

Список литературы

1. *Robbins A.* Bash Pocket Reference. — O’Reilly Media, 2016.

Ответы на контрольные вопросы

1. Найдите синтаксическую ошибку в `while [$1 != "exit"]`. Нет пробелов вокруг `[` и `]`: нужно `[ "$1" != "exit" ]`.

2. Как выполняется конкатенация строк в bash? Просто пишутся подряд: `c="$a$b"`.

3. Что делает утилита `seq` и чем её можно заменить? Выводит числа по порядку (`seq 1 10`); замена — `{1..10}` или `for ((i=1;i<=10;i++))`.

4. Чему равно `$((10/3))`? 3 — деление целочисленное.

5. Чем zsh отличается от bash? Более гибкие автодополнение и глобы, поддержка плагинов (oh-my-zsh), другой синтаксис массивов.

6. Корректен ли синтаксис `for ((a=1; a <= LIMIT; a++))`? Да, корректный C-подобный цикл.

7. Сравните bash с языками программирования (плюсы/минусы). Плюс — быстрая автоматизация системных задач без компиляции; минус — низкая производительность и плохая читаемость сложной логики.